our shielding · Your smart contracts, our shielding · Your smart c

shieldify

Pear Protocol

Vault

SECURITY REVIEW

Date: 10 March 2026

CONTENTS

1. About Shieldify **3**

2. Disclaimer **3**

3. About Pear Protocol – Vault **3**

4. Risk classification **4**

4.1 Impact 4

4.2 Likelihood 4

5. Security Review Summary **4**

5.1 Protocol Summary 4

5.2 Scope 4

6. Findings Summary **5**

7. Findings **5**

1. About Shieldify

Positioned as the first hybrid Web3 Security company, Shieldify shakes things up with a unique subscription-based auditing model that entitles the customer to unlimited audits within its duration, as well as top-notch service quality thanks to a disruptive 6-layered security approach. The company works with very well-established researchers in the space and has secured multiple millions in TVL across protocols, also can audit codebases written in Solidity, Vyper, Rust, Cairo, Move and Go.

Learn more about us at shieldify.org.

2. Disclaimer

This security review does not guarantee bulletproof protection against a hack or exploit. Smart contracts are a novel technological feat with many known and unknown risks. The protocol, which this report is intended for, indemnifies Shieldify Security against any responsibility for any misbehavior, bugs, or exploits affecting the audited code during any part of the project's life cycle. It is also pivotal to acknowledge that modifications made to the audited code, including fixes for the issues described in this report, may introduce new problems and necessitate additional auditing.

3. About Pear Protocol - Vault

Pear Protocol represents an innovative solution designed to streamline and enhance the efficiency of on-chain pairs trading. It enables users to execute leveraged long and short positions within a single transaction, addressing the complexities and inefficiencies traditionally associated with pair trading in cryptocurrencies. By integrating a variety of on-chain trading engines alongside a dedicated user interface and experience, Pear Protocol simplifies the process of initiating simultaneous long and short positions in correlated assets, such as going long on BTC while shorting ETH with leverage.

This liquidity-agnostic platform offers deep liquidity access and flexibility in managing trading parameters, and mitigates the custody and trust issues found in centralized exchanges by ensuring traders retain asset custody. Beyond simplifying trading executions, Pear Protocol extends its utility with a tokenized trading system, allowing trading positions to be represented as ERC-721 tokens for increased composability within DeFi ecosystems. With features emphasizing simplicity, flexibility, scalability, optionality, and ease of use, Pear Protocol aims to revolutionize pair trading, making it more accessible and efficient while fostering broader decentralized finance trading solutions adoption.

Learn more about Pear's concept and the technicalities behind it here.

Pear Vault System Summary

This codebase implements the **Pear Vault** system, a decentralized asset management protocol built on Hyperliquid. The system allows whitelisted users (Leaders) to create individual investment vaults. Depositors can then fund these vaults to have the Leader trade with their assets on the Hyperliquid platform.

The architecture is designed to be flexible and upgradeable, utilizing a factory pattern for vault creation and adhering to the `ERC4626` tokenized vault standard.

Severity	Impact: High	Impact: Medium	Impact: Low
Likelihood: High	Critical	High	Medium
Likelihood: Medium	High	Medium	Low
Likelihood: Low	Medium	Low	Low

4. Risk Classification

4.1 Impact

- **High** – results in a significant risk for the protocol's overall well-being. Affects all or most users
- **Medium** – results in a non-critical risk for the protocol affects all or only a subset of users, but is still unacceptable
- **Low** – losses will be limited but bearable – and covers vectors similar to griefing attacks that can be easily repaired

4.2 Likelihood

- **High** – almost certain to happen and highly lucrative for execution by malicious actors
- **Medium** – still relatively likely, although only conditionally possible
- **Low** – requires a unique set of circumstances and poses non-lucrative cost-of-execution to rewards ratio for the actor

5. Security Review Summary

The security review lasted 4 days with a total of 64 hours dedicated to the audit by the Shieldify team.

Overall, the code is well-written. The audit report identified two Low severity issues. They're related to insufficient input validation and unsafe token handling patterns that can lead to misconfiguration or unintended state changes.

This is the second security review that Shieldify has conducted on Pear Protocol Vaults, with this audit focused primarily on the newly-added functionality.

5.1 Protocol Summary

Project Name	**Pear Protocol Vault – Extended**
Repository	pear-vault-smartcontracts
Type of Project	DEX, On-Chain Pairs Trading
Security Review Timeline	4 days
Review Commit Hash	0c91581c9c393a5799f76a685d675d9069e1fbc2
Fixes Review Commit Hash	c5e4a7f090ec32d16c95a261d9e81617cea68b66

5.2 Scope

The following smart contracts were in the scope of the security review:

File	nSLOC
src/PearVaultFactory.sol	116
src/Comptroller.sol	234
src/PearVault.sol	649
src/libraries/HyperliquidHelper.sol	66
Total	1065

6. Findings Summary

The following number of issues have been identified, sorted by their severity:

- **Low** issues: **2**

ID	Title	Severity	Status
[L-01]	Missing Upper-Cap and No-Op Validation in `setAgentGasFee()` Allows Arbitrary and Redundant Updates	Low	Fixed
[L-02]	Unsafe ERC20 approve Usage in `depositToFelixVault()`	Low	Fixed

7. Findings

[L-01] Missing Upper-Cap and No-Op Validation in `setAgentGasFee()` Allows Arbitrary and Redundant Updates

Severity

Low Risk

Description

The `setAgentGasFee()` function allows the contract owner to update `_agentGasFee` without any validation on the new value. There is no upper bound check to prevent setting an excessively high gas fee, nor is there a validation to prevent updating the fee to the same value as the current one.

While the function is restricted by `onlyOwner`, the absence of validation can still introduce configuration risks. An excessively high gas fee could unintentionally disrupt protocol operations, overcharge users, or render certain interactions economically unviable. Additionally, allowing redundant updates where `newFee` equals the current `_agentGasFee` results in unnecessary state changes and event emissions, which can create misleading off-chain signals and reduce operational clarity.

Location of Affected Code

File: src/Comptroller.sol#L321-L326

```
function setAgentGasFee(uint256 newFee) external override onlyOwner {
    uint256 oldFee = _agentGasFee;
    _agentGasFee = newFee;

    emit AgentGasFeeUpdated(oldFee, newFee);
}
```

Impact

- Break deposit flows.
- Causes excessive ETH extraction from users.
- Lead to denial-of-service scenarios.
- Create economic inconsistencies in vault operations.

Recommendation

Introduce validation checks to ensure that the new fee remains within a reasonable upper bound defined by protocol requirements. Additionally, add a condition to prevent redundant updates by reverting if `newFee` is equal to the current `_agentGasFee`.

Team Response

Fixed.

[L-02] Unsafe ERC20 Approve Usage in `depositToFelixVault()`

Severity

Low Risk

Description

The function `depositToFelixVault()` and `_sendFundsToSystem()` use a direct ERC20 `approve()` call.

This is unsafe because: - Some ERC20 tokens require allowance to be set to 0 before updating it. - It does not check for return values, which may lead to silent failures for non-standard ERC20 implementations.

The contract imports IERC20 but does not use OpenZeppelin's SafeERC20, which provides safe wrappers.

Location of Affected Code

File: src/PearVault.sol#L953

```
function depositToFelixVault(uint256 assets) external override onlyOwner
   nonReentrant validAmount(assets) {
  // code
  vaultToken().approve(vaultConfig.felixVault, assets);
  // code
}
```

File: src/PearVault.sol#L1047-L1048

```
function _sendFundsToSystem( address token, address agentWallet, address
    sysAddr, uint256 amount ) internal {
  // code
  tokenContract.approve(address(coreWallet), amount);
  // code
```

Impact

Using raw approve instead of `safeApprove()` may cause compatibility issues with non-standard ERC20 tokens and can result in unexpected reverts if allowance is not first set to zero.

Recommendation

Replace the direct approve call with `SafeERC20.safeApprove()`.

Team Response

Fixed.

our shielding · Your smart contracts, our shielding · Your smart c

shieldify

Thank you!